

1-3

let $x.size$ = total number of keys in subtree rooted at x , and $x.size$ should be adjusted after every insertion and deletion

COUNT-STUDENTS(T, l, r)

return COUNT-SUBTREE($T.root, r$) - COUNT-SUBTREE($T.root, l - 1$)

COUNT-SUBTREE(x, k)

cnt = 0

i = 1

while $i \leq x.n$ and $k \geq x.key[i]$

if not $x.leaf$

cnt += $x.c[i].size$

cnt += 1

i++

if $x.leaf$

return cnt

return cnt += COUNT-SUBTREE($x.c[i], k$)

Explanation: adding a field "size" allows the track of the size of each subtree, so we can access the number of integers that is less than or equal to k in $O(1)$ time. Then we build a helper function to recursively count the number of keys that is less than or equal to r and subtract it by the number of keys less than or equal to $l - 1$ (since the keys are integers, counting $k \leq l - 1$ is equivalent to counting $k < l$).

Time complexity: In the worst-case scenario, the computer will have to read through every key in a node, which owns $2t - 1$ keys at most, and will have to go all the way down to the leaf, which takes $O(\log_t n)$ time. Therefore, the algorithm takes $O(t \log_t n)$ time to complete the job

Space complexity: the bottleneck is the recursive function stack, which will be called $O(\log_t n)$ times and take $O(\log_t n)$ space in the worst-case scenario, therefore satisfying the time constraint.

1-4

B-TREE-INSERT-2(T, k)

$r = T.root$

```

INSERT-MAYBE-FULL(r, k)
    if r.n == 2t
        SPLIT-ROOT-2(r)
INSERT-MAYBE-FULL(x, k)
    i = x.n
    if x.leaf
        while i >= 1 and k < x.key[i]
            x.key[i + 1] = x.key[i]
            i--
        x.key[i + 1] = k
        x.n++
        DISK-WRITE(x)
    else
        while i >= 1 and k < x.key[i]
            i--
        i++
        DISK-READ(x.c[i])
        INSERT-MAYBE-FULL(x.c[i], k)
        if x.c[i].n == 2t
            SPLIT-CHILD-2(x, i)

```

Explanation: the algorithm insert a key with a helper function. It does the insertion all the way down at a leaf node, and then maintain the overflow issue on the way back. If a child $x.c[i].n == 2t$, the algorithm calls SPLIT-CHILD-2 on x to split it. If $T.root.n == 2t$, the function calls SPLIT-ROOT in the end.

2-1(a)

(i) 2

(ii) 48

2-1(b)

(i) 6

(ii) 192

2-3

G = prerequisite graph

EARLIEST-GRAD-TIME(G)

memo = create an array with length n and initialize every element to 0

DFS(G)

max_memo = 0

for i from 1 to n

if memo[i] > max_memo

max_memo = memo[i]

return max_memo

DFS(G)

for each vertex u belongs to G.V

u.color = WHITE

time = 0

for each vertex u belongs to G.V

if u.color == WHITE

DFS-VISIT(G, u)

DFS-VISIT(G, u)

u.color = GRAY

max_v_time = 0

for each vertex v belongs to G.adj[u]

if v.color == WHITE

DFS-VISIT(G, v)

```

    if memo[v.id] > max_v_time
        max_v_time = memo[v.id]

    u.color = BLACK

    memo[u.id] = T[u.id] + max_v_time

    time++

```

Idea: this algorithm computes the earliest graduation time by finding the longest weighted path in the prerequisite DAG, where the weight of course i is $T[i]$. Since any number of courses can be taken in parallel, the only thing that delays graduation is a chain of courses that must be taken one after another. Therefore, the earliest graduation time is exactly the maximum total duration over all prerequisite chains. To compute this, we run dfs on the graph. For each course i , $\text{memo}[i]$ stores the minimum time needed to finish all courses on the longest path starting from i . We finally return the maximum value of memo as the answer.

Time complexity: the time complexity for the dfs algorithm is identical to that of the original version, which takes $O(n + m)$ time. Find the maximum value in the memo array takes $O(n)$ time. Overall, the time complexity is $O(n + m)$, which satisfies the constraint.

Space complexity: In the worst-case scenario, the graph is a linked list. The space required for the recursive call stack will be $O(n)$. The extra space needed for memo array in whatever case is $O(n)$. Overall, the space complexity is $O(n)$, which satisfies the constraint.

2-4(a)

suppose the route of a knight is: start $\rightarrow \dots \rightarrow u \rightarrow a \rightarrow b \rightarrow u \rightarrow \dots \rightarrow \text{meet}$. At the point when it visits u the second time, more pawns have been placed compared to the first time it visit u . When the knight reaches u the second time, the board is more blocked if it can still follow the rest of the route from that second visit to the meeting square, then that same suffix path would also have been feasible from the first visit to u when fewer pawns had been placed. Hence, the loop $u \rightarrow a \rightarrow b \rightarrow u$ is completely wasting time. If a knight can visit a spot more than once, it will always have a better solution, which is to skip the loop and move on, so in any optimal solution, a knight will never visit the same cell more than once.

2-4(b)

EARLIEST-MEETING-DAY(n, m, P)

```

     $N = n * m$ 

```

```

    for each cell  $c$  on the board

```

```

         $\text{ban}[c] = -1$ 

```

for t from 1 to N

$\text{ban}[P[t]] = t$

if $(n + m) \% 2 \neq 0$

 return -1

distA = BFS((1, 1), ban, n, m)

distB = BFS((n, m), ban, n, m)

ans = +INF

for each cell c on the board

 if $\text{distA}[c] \neq \text{INF}$ and $\text{distA}[c] == \text{distB}[c]$

$\text{ans} = \min(\text{ans}, \text{distA}[c])$

if $\text{ans} == \text{INF}$

 return -1

return ans

BFS(s, ban, n, m)

for each cell c on the board

$\text{dist}[c] = \text{INF}$

if $\text{ban}[s] \leq 1$

 return dist

$\text{dist}[s] = 0$

Q = createQueue()

ENQUEUE(Q, s)

while Q is not empty

$u = \text{DEQUEUE}(Q)$

$d = \text{dist}[u]$

 if $\text{ban}[u] \leq d + 1$

 continue

$u_neighbor = \text{all combinations of points } (u.\text{row} \pm 2, u.\text{col} \pm 1) \text{ or } (u.\text{row} \pm 1, u.\text{col} \pm 2)$

for each cell v in $u_neighbor$

 if v is outside the board

```

        continue

    if dist[v] != INF

        continue

    if ban[v] <= d + 1

        continue

    dist[v] = d + 1

    ENQUEUE(Q, v)

return dist

```

Idea: the algorithm models the board as an undirected graph where each cell is a vertex and an edge connects two cells where a knight can possibly move to. For each cell c , let $\text{ban}[c]$ store the day on which c is occupied by a pawn. According to part(a), a knight won't be visiting a cell more than once in an optimal solution, so it suffices to compute the earliest day it can reach each cell. $\text{distA}[c]$ and $\text{distB}[c]$ stores the earliest day the knight A and B to reach cell c , respectively. For each knight, bfs explores the cells in increasing number of days. When a cell c is first assigned $\text{dist}[c] = d$, this means the knight can first land on c after exactly d moves. The final answer will be the minimum d such that $\text{distA}[c] == \text{distB}[c]$.

Time complexity: the time complexity of bfs is $O(|V| + |E|) = O(mn + 8mn) = O(mn)$ in this case

Space complexity: arrays ban , distA , distB take $O(mn)$ space, which is the bottleneck of space complexity. Hence, this algorithm takes $O(mn)$ space.

2-5

ROOK-SURVIVE(n, m, P)

```

left = 1

right = n * m

last_success_day = 0

while left <= right

    mid = floor(left + right / 2)

    if CAN-CROSS(n, m, P, mid)

        last_success_day = mid

        left = mid + 1

else

```

```
right = mid - 1
```

```
return last_success_day
```

```
CAN-CROSS(n, m, P, day)
```

```
grid = create a 2D array with n rows * m cols and initialize every element to 0
```

```
for t from 1 to day
```

```
    grid[P[t].row][P[t].col] = 1 // 1 means block by a pawn
```

```
Q = createQueue()
```

```
for r = 1 to n
```

```
    v = (r, 1)
```

```
    if grid[r][1] == 0
```

```
        ENQUEUE(Q, v)
```

```
        grid[r][1] = 2 // 2 means visited
```

```
while Q is not empty
```

```
    u = DEQUEUE(Q)
```

```
    if u.col == m
```

```
        return true
```

```
    v = the four combinations of [(u.row ± 1, u.col), (u.row, u.col ± 1)]
```

```
    for each cell c in v
```

```
        if 1 ≤ c.row ≤ n and 1 ≤ c.col ≤ m
```

```
            if grid[c.row][c.col] == 0
```

```
                ENQUEUE(Q, c)
```

```
                grid[c.row][c.col] = 2
```

```
return false
```

Idea: The algorithm uses binary search to find the answer. Since the maximum possible answer is $m * n$, we initialize right to $m * n$. If a day is crossable, we look for a larger day. Otherwise, we look for a smaller day. We then use bfs to check if a day is crossable. We first initialize a 2D array to model the grid and place all the pawn that should be place on the day. Then we run the bfs. Since the valid moves for rook will always be the adjacent cells (a long move can be broken down into single moves). If we reach column m, that day is crossable. We finally return the largest crossable day as the answer.

Time complexity: the time spent by the CAN-CROSS function includes the bfs algorithm, which takes $O(|V| + |E|) = O(mn + 4mn) = O(mn)$ time. Also, it takes $O(mn)$ time to initialize the grid, and $O(\text{day}) = O(mn)$ time to mark the blocked cells. So the time complexity for CAN-CROSS function is $O(mn)$. The binary search in the ROOK-SURVIVE function take will call CAN-CROSS $O(\log(mn))$ time, resulting in an overall time complexity of $O(mn * \log(mn))$

Space complexity: the bottleneck is the model grid and the queue created in the CAN-CROSS function, which takes $O(mn)$ space because of the 2D array and another $O(mn)$ space for the queue since it can hold $m * n$ points at most, resulting in an overall $O(mn)$ space complexity.